

Parallel Image Blurring using Data-Level and Task-Level Decomposition in C++

This project takes a look at how parallel image processing using C++ multi-threading with a focus on faster performance through decomposition methods. The exercise has started with the implementation of data-level parallelism for blurring a single image, and then it was moved to task-level parallelism where dealing with an entire folder of images by dividing the process into several parts. The C++17 language was used to write the code for the project, OpenCV library was utilized for image processing and the C++ Standard Library was used for multithreading and file system operations.

Phase 1 – Parallel Blurring of a Single Image (Data-Level Decomposition, Geometric Decomposition)

In the first version of the project, I implemented the effect of a box blur on a single color image using a user-specified kernel size. The image was fragmented into different horizontal slices, where each part was taken by a thread to process further. Every thread independently did the convolution over its unique rows, and all of them used a square averaging kernel but retained the square form (e.g., 3×3, 5×5, up to 21×21). Such a decomposition is data-parallel since threads conduct operations on the non-overlapping parts of the same image.

Then measuring the execution time of the multi-threaded version and the single-threaded version. The created image is being saved after the affirming process, and from this point, the effect of performance is represented as a significant improvement, while dealing with huge images and various kernels.

Phase 2 – Folder-Based Parallel Image Blurring (Task-Level Decomposition, each thread handles its own image)

The next stage of the project was to extend and be able to handle Batch processing of several images in a folder. In this process, each picture is considered a separate task and a group of worker threads are created to process them at the same time. This approach permits task-level parallelism from sharing the input workload (images) between different threads, each performing the blur function as explained in phase one.

The size of the kernel and the quantity of threads can be chosen by the user at runtime. The software is designed so that it can read from the "Images" folder (which should be in the same directory as the executable), and it saves the results in two different folders:

- Blurred_Single for single-threaded batch execution
- Blurred_Multi for multithreaded batch execution

The creation of the folders are automatic and timestamping of the results are shown for performance comparison.

Results and Observations

A kernel size of 15 and a dataset of very high-resolution images are used. The system, illustrates quite remarkable speed, indicated the performance measurement:

- Single-threaded processing time: 502.359 seconds
- Multi-threaded (8 threads) processing time: 168.015 seconds

The figure of about 3 time more speed, showing off the effect of Multi-threading. The performance gap widens as image size or kernel size increases, highlighting the scalability of the task-level model.

This successfully achieved that both data decomposition and task decomposition were efficiently used for parallelizing image processing workloads. The first part emphasizes that the parallelization was done by dividing a single image's pixel data into parts, while in the second part, the effort concentrated on a model that was adjusted to manage multiple independent image-processing tasks. This composition model has a lot in common with the structure of many actual systems.